Advanced Scientific Research Projects LLP
ТОО «Перспективные Научно-Исследовательские Разработки» · BIN 240140033296
Republic of Kazakhstan · asrp.tech

Arcanum Landing Engine — AI Course-Landing Generation for an Online School

Delivered by: ASRP · **Product:** Arcanum 12th (arcanum12th.education) · **Role:** Full-cycle design & build (agent · API · authoring editor · public renderer) · **Engagement type:** Own product · **Domain:** AI content automation for a multilingual education platform

1. Executive summary

Arcanum Landing Engine is ASRP's **full-cycle, AI-assisted system for building course landing pages** for its own online school, **Arcanum 12th** (live at arcanum12th.education, 14 courses, 14 UI locales). It turns structured course data into a published marketing page with almost no manual layout work — and leaves a human fully in control of the result.

The cycle spans four cooperating parts, all built by ASRP: a **GraphQL backend** (`arcanum12th-api`) assembles a "build context" from real course data and a structural contract; a **headless AI agent** (`agent-landing-builder` , FastAPI + OpenAI) plans a layout, writes the copy, validates it against a block catalog, and self-repairs — emitting a JSON payload of typed sections; a **Nuxt authoring editor** (`arcanum12th-author-web`) lets an author refine that payload visually — reorder, add blocks from a ⌘K palette, edit each block inline, tune light/dark theme tokens, switch language; and a **public Nuxt renderer** (`website`) maps each section type to a Vue component and paints the live page. The unit of exchange throughout is a single **landing payload**: `{ sections: [{ id, type, props }], themeStyles }` .

It runs in production, generating the live course landing pages at arcanum12th.education.

2. Positioning & problem

An online school with a growing course catalog needs a **marketing landing page per course** — hero, program, stats, pricing, FAQ, contacts — in several languages. Hand-building each one is slow and drifts in quality; generating one from a single "make me a landing" prompt yields unstructured HTML nobody can safely edit or translate.

Arcanum's answer treats a landing as **structured data, not markup**, and splits the job across a machine and a human. The machine (the agent) is good at drafting a complete, on-catalog layout from facts; the human (the author) is good at judgment — the final wording, ordering, imagery, and brand feel. By making the hand-off a typed **section payload** rather than HTML, the same artifact can be generated, refined in an editor, translated per language, stored, audited, and rendered — each stage owning its concern, none of them re-parsing the others' output.

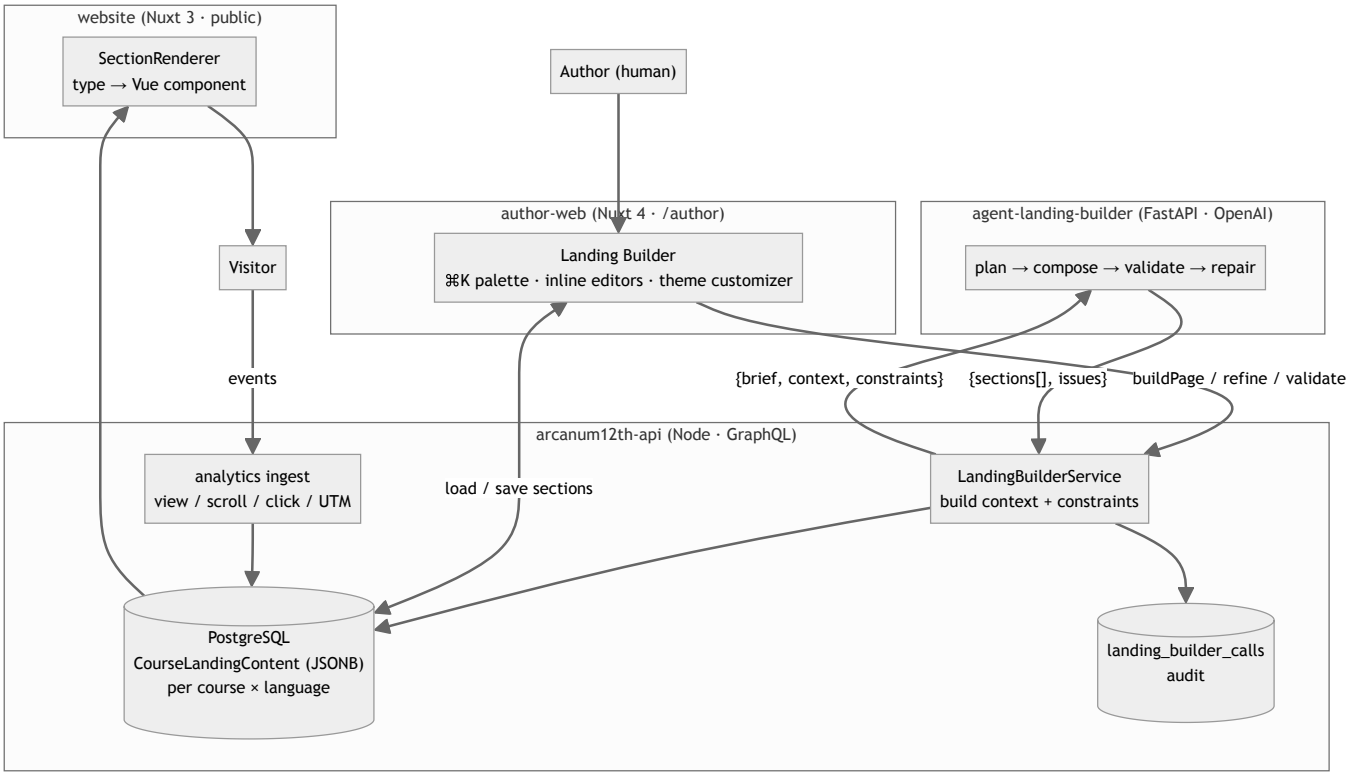
3. What was built

Four cooperating components, all ASRP-built:

- **`agent-landing-builder` — the AI generator.** A FastAPI microservice (OpenAI `gpt-4o-mini`) exposing `/build-page`, `/refine-page`, `/validate-page`. It plans a layout, composes copy into each block's props, validates against a block catalog, and runs a bounded repair loop — with a deterministic fallback for every LLM stage so it works with no API key.
- **`arcanum12th-api` — the backend & orchestrator.** A Node.js GraphQL API (Apollo Server 4, Sequelize/PostgreSQL) with four application overlays (public / admin / author / partner). Its `LandingBuilderService` assembles the build context and structural contract from real course data, calls the agent, normalizes and persists the payload, and logs every call for audit. It also ingests landing analytics (views, scrolls, clicks, UTM).
- **`arcanum12th-author-web` — the authoring editor.** A Nuxt 4 admin app served at `/author`. Beyond full content management (articles, courses, lessons), its flagship is the **Landing Builder**: a ⌘K add-section palette (~60 section types), per-section inline editors, a light/dark theme customizer, per-language switching, templates, and import/export JSON.
- **`website` — the public renderer.** A Nuxt 3 site that reads the stored payload for the visitor's locale and maps each `section.type` to a Vue component, injecting `themeStyles` as CSS variables and streaming analytics back to the API.
- **Wider ecosystem.** The same platform (shared GraphQL API) also powers a **Telegram-bot** channel — course-catalog browsing, an education flow, an AI dream-diary, and payments — a second consumer of the same course data alongside the public website.

4. Architecture

The four parts form a loop around one payload. The API is the hub: it owns the contract and the storage; the agent drafts; the editor refines; the site renders.



Component	Responsibility
author-web (Nuxt 4)	Authoring UX: the Landing Builder editor, the block catalog + default props, theme customization, per-language switching, and the “generate with AI” trigger.
arcanum12th-api (Node/GraphQL)	The hub: assembles the build context + structural contract, orchestrates the agent, normalizes & persists the payload (JSONB per course×language), audit logging, analytics ingest.
agent-landing-builder (FastAPI/OpenAI)	The generator: layout planning, copywriting, catalog validation, and bounded self-repair — deterministic fallbacks throughout.
website (Nuxt 3)	Public rendering: `type → component` registry, theme tokens as CSS variables, analytics events.

5. The generation pipeline

Inside the agent, `build_landing()` runs four stages, each with a deterministic fallback so the service degrades gracefully when OpenAI is unavailable (gated by `_should_use_llm()`):

1. **Planner** — chooses an ordered section plan (`{family, type, required, …}`) from the block catalog, honouring the API’s constraints (`must_include` , `preferred_order` , `max_sections`). LLM prompt `@landing-planner` ; deterministic fallback `plan_sections()` .
2. **Composer** — writes each planned section’s props from the supplied course context. LLM prompt `@landing-copywriter` ; deterministic fallback `compose_payload()` .

3. **Validator** — checks the payload against the catalog + constraints (`validate_payload()`), with an LLM quality evaluator (`@landing-evaluator`) and a deterministic quality score.
4. **Repair** — an LLM reflection pass (`@landing-reflector`) plus deterministic `repair_payload()` . The orchestrator **only keeps a repair if it reduces the issue count** (`if len(repaired_issues) <= len(issues)`) — bounded, never worse.

The backend shapes the agent's freedom with **layout hints**: per block, a `mode` of `llm_content` (the model writes it) or `deterministic_props` (filled from context, e.g. pricing, stats), plus `fill_from_context` sources. So facts stay factual and only the persuasive copy is model-generated.

6. The landing contract & block catalog

The contract is one payload — `{ sections: [{ id, type, props }], themeStyles: { light, dark } }` — and a vocabulary of section `type` s grouped into families (Header, Hero, About, Features, Stats, Timeline, Steps, Blog, Pricing, FAQ, Social proof, CTA, Contact, Map, Footer, and platform-aware blocks like dynamic course cards). Many families have **versioned variants** (e.g. `stats-v1..v6` , `features-v1..v6` , `timeline-v1..v4` , `faq-v1..v4`), so a layout can vary visually without changing the data shape.

A candid engineering note: this contract is **replicated by convention in three places** — the agent's catalog (`agent/catalog/defaults/**`), the editor's catalog (`useLandingSections.ts` + `landingDefaultProps.ts` + `landingSectionComponents.ts`), and the public renderer's `type → component` map (`SectionRenderer.vue`). The wire type is opaque JSON, so nothing machine-checks it end-to-end. The commit **"Sync catalog with frontend landing contracts"** (agent, 2026-06-09) is exactly the moment the agent's catalog was pinned to the frontend's real block/prop shapes. The trade-off is real: if a section `type` exists in the editor but not the renderer, the public page silently renders nothing for it — a known drift risk inherent to a convention-enforced contract.

7. Progress & iteration

The engine visibly improved over successive runs. The `artifacts/` folder captures two parallel v1→v7 series (build responses and extracted payloads) from **2026-06-08 → 06-09**. The section count holds steady at 12, so the gains show in **structure and prop richness** — populated prop-keys climb from ~139–153 early to **170 (v5) and 179 (v7)**, empty props fall to single digits mid-series, sections get reordered (contact up, FAQ earlier), and later versions swap in refined variants (`features-v2` , `faq-v1`). That is the repair/evaluation loop tightening output, run over run.

The git history tells the same story as milestones:

Phase	What landed
Scaffold & core pipeline (06-06)	Initial builder, LLM planning pipeline, tests, curl examples
Ops & catalog metadata (06-06→07)	Docker CI, quality roadmap, enriched catalog, semantic validation + golden tests, request tracing
Context genericization (06-07→08)	Generic/raw course context input, structured course context
Pricing determinism & catalog split (06-08)	Pricing-aware planning, deterministic pricing, catalog split into manifest families
Layout planning + LLM eval/reflection (06-08→09)	Abstract layout plan, LLM evaluation, reflection prompts, positioning memo
Contract cleanup (06-08→09)	Stop generating theme styles, backend-driven mandatory sections, documented layout-driven contract
Catalog sync & hardening (06-09→12)	Catalog linting + benchmark harness, planner/copywriter split, **sync catalog with frontend contracts**, tighter copy heuristics

The surrounding stack matured on the same timeline: the API's landing feature (`CourseLandingContent`) landed **2026-05-13**, and author-web's Landing Builder epic ran **May–Jul 2026**.

8. Stack

Layer	Choice
Agent	Python 3.13, FastAPI + Uvicorn, Pydantic v2, OpenAI (`gpt-4o-mini` , env-driven, `temperature 0` , JSON mode)
Backend API	Node.js, Apollo Server 4 + Express, GraphQL, Sequelize + PostgreSQL, Redis (sessions/cache), pg-boss; JWT/API-key auth; four application overlays
Authoring editor	Nuxt 4 (Vue 3, TypeScript, SSR/Nitro), Pinia, shadcn-vue + Tailwind v4, Apollo Client + GraphQL codegen, TipTap v3, xstate, 14 locales
Public renderer	Nuxt 3 (Vue 3), Tailwind v4, Apollo Client, i18n; `type → component` section registry, theme tokens as CSS variables
Ops	Docker Compose, images to GHCR, Nginx + certbot (TLS), PM2 cluster; submodule monorepo (`arcanum12th-core`)

9. Data & interfaces

- **Storage.** `CourseLandingContent { courseId, languageId, sections: JSONB }` — **one payload per (course, language)**, soft-deleted (paranoid). The website eager-loads it via the public `course` query; there is no separate “get landing” endpoint.
- **GraphQL surface.** Author overlay: `courseLandingContentBuildPage / refinePage / validatePage`, `courseLandingContentBuildContext` (preview the assembled context), plus CRUD `courseLandingContentCreate / Update / Delete`. Public overlay: analytics mutations `courseLandingView / Click / Scroll / Utm`. Admin overlay: `landingBuilderCalls` audit query.
- **Agent API.** `POST /build-page`, `POST /refine-page` (requires `previous_draft`), `POST /validate-page`, `GET /health`.
- **Audit.** Every agent call is written to `landing_builder_calls` (request, response, issues, context, timing, status) — a full generation trail.

10. Reliability & operations

- **Graceful degradation.** Every LLM stage has a deterministic fallback; without an OpenAI key the agent still emits a valid on-catalog payload.
- **Bounded repair.** A repair is kept only if it reduces the issue count — the loop can’t make a page worse.
- **Human in the loop.** The AI output is a *draft*; nothing publishes until an author saves in the editor.
- **Auditability.** `landing_builder_calls` records every generation; `X-Request-Id` tracing runs through the agent.
- **Analytics.** Views, scrolls, clicks, and UTM stream from the public page back into the API.
- **Deployment.** Docker Compose behind Nginx + certbot TLS; images in GHCR; PM2 cluster inside the app containers; Postgres 17 + Redis. The agent runs as an internal-only service the author-API depends on. The stack is composed as a submodule monorepo with `manifest.json` pinning each submodule SHA.

11. See it live

- **Live product:** arcanum12th.education — the school, 14 courses, multiple locales. Course landing pages are the rendered output of this engine.
- **Author panel:** arcanum12th.education/author — the Landing Builder editor and the wider authoring platform (rich Markdown editor, multi-format articles/courses/lessons, AI assistant, 500+ authors).

Screenshots below show the authoring side (the AI-drafted section payload being refined) and the rendered public landings.

> *[screenshot / скриншот: `assets/arcanum-author-web-builder.png` — Landing Builder: section list + live preview / список секций + живой превью]*
>
> *[screenshot / скриншот: `assets/arcanum-author-hero.png` — Hero section inline editor (CTA + stats) / инлайн-редактор Hero (CTA + статы)]*
>
> *[screenshot / скриншот: `assets/arcanum-author-about.png` — About section editor (image + principle cards) / редактор About (картинка + карточки-принципы)]*

> *[screenshot / скриншот: `assets/arcanum-author-timeline.png` — Timeline: course program by module / программа курса по модулям]*
>
> *[screenshot / скриншот: `assets/arcanum-course-landing.png` — rendered course landing (public) / отрендеренный лендинг курса (публично)]*

12. Status & roadmap

Status: production. The API, author-web, and public website are deployed and live at arcanum12th.education across 14 courses. The agent is a well-structured engine at `0.1.0` — strong test coverage (API, LLM pipeline, live smoke, semantic validation, golden briefs, catalog lint, evaluation harness), CI Docker builds, and request tracing, with integration-hardening (auth/rate-limiting on the agent endpoints) still on the roadmap. Natural next steps: a single shared catalog package to remove the three-way contract drift, per-type prop schemas (today `props` is free-form), and closing the editor↔renderer variant gap.

13. What ASRP delivered

ASRP designed and built the entire system end to end — all four components (`agent-landing-builder` , `arcanum12th-api` landing feature + `LandingBuilderService` , `arcanum12th-author-web` Landing Builder, and the `website` renderer), the shared landing-payload contract and block catalog, the deployment topology, and the AI generation pipeline — as its own product for its own online school.

Own-product case study prepared by ASRP for portfolio use. Arcanum 12th and arcanum12th.education are ASRP's own product and are named accordingly. Figures reflect the system as of 2026-07; the public site is live. Source repositories are private; capability-level detail only.